

TECNOLÓGICO NACIONAL DE MÉXICO INSTITUTO TECNOLÓGICO DE VERACRUZ

Práctica 1

Martínez Palacios Luis Felipe
22020051

Bases de Datos NOSQL

23 de septiembre del 2026



Objetivos

- Desplegar un contenedor de MongoDB utilizando Podman en Windows.
- Configurar un volumen administrado por Podman para garantizar la persistencia de los datos.
- Verificar el correcto funcionamiento del servidor MongoDB.
- Insertar información de prueba dentro de una base de datos.
- Comprobar que la información permanece disponible después de detener y volver a iniciar el contenedor.

Preparación del entorno

Para realizar el despliegue se utilizó **Podman en Windows**, ejecutando los comandos desde PowerShell.

Primero se verificó que Podman estuviera correctamente instalado mediante:

```
PS C:\Users\mplf2> podman --version
podman version 6.0.2
```

Posteriormente se comprobó el estado del entorno de Podman:

```
podman version 6.0.2
PS C:\Users\mplf2> podman info
Client:
  APIVersion: 6.0.2
  Built: 1784689850
  BuiltTime: Tue Jul 21 20:57:38 2026
  GitCommit: b28edb9ad78ce4317dc762ee9ce0a6d081d154e9
  GoVersion: go1.26.5
  Os: windows
  OsArch: windows/amd64
  Version: 6.0.2
Host:
  arch: amd64
  buildahVersion: 1.44.1
  cdiSpecDirs:
    - /etc/cdi
    - /var/run/cdi
  cgroupControllers:
    - cpu
    - io
    - memory
    - pids
  cgroupManager: systemd
  cgroupVersion: v2
  common:
    package: common-2.2.1-2.fc44.x86_64
    path: /usr/bin/common
    version: 'common version 2.2.1, commit: '
  cpuUtilization:
    idlePercent: 99.62
    systemPercent: 0.18
    userPercent: 0.2
  cpus: 16
  databaseBackend: sqlite
  distribution:
    distribution: fedora
    variant: podman-machine-os
    version: "44"
  eventLogger: journald
  freeLocks: 2038
  hostname: Orito
```

También se verificó la máquina de Podman:

```
PS C:\Users\mplf2> podman machine ls
NAME                VM TYPE   CREATED      LAST UP           CPUS    MEMORY   DISK SIZE
podman-machine-default* wsl       38 minutes ago Currently running  8       2GiB     100GiB
PS C:\Users\mplf2> |
```

En caso de que la máquina no estuviera inicializada, se puede crear mediante:

podman machine init

Y posteriormente iniciarla:

podman machine start

Una vez iniciado el entorno, Podman quedó preparado para ejecutar contenedores Linux.

Creación del volumen para la persistencia

Para evitar que los datos de MongoDB dependieran directamente del ciclo de vida del contenedor, se creó un volumen administrado por Podman.

Se utilizó el siguiente comando:

```
PS C:\Users\mplf2> podman volume create mongodb_data
mongodb_data
```

Para comprobar que el volumen fue creado correctamente:

```
PS C:\Users\mplf2> podman volume ls
DRIVER      VOLUME NAME
local       9785e2eafb6bbceb963d7422dc1aefa5e2a0c10917b04e38014ce40f33b484b1
local       bf73d0defcf106a9fc2aff7881e45179f2ed638aeae20bc279b141caeca2e55d
local       a8e0c7d01b40d9b876302a118eda622195b65e5d9b71d4893d06a38600d1b74f
local       mongo_data
local       e1eb01594fda9e0f67f5a7c7e76eadc80e443a791572380d3129e98b97394d
local       6a637d83e289973c7efa5451ca8e1f60a9e363841b40698d05b5242636cc9736
local       620b53938c281ff8a91c533ae6846aceb14ce7197425c4ca2d4b28b73db6e315
local       mongodb_data
```

El resultado mostró el volumen:

DRIVER	VOLUME NAME
--------	-------------

local	mongodb_data
-------	--------------

El volumen será utilizado posteriormente para almacenar los archivos de MongoDB en:

/data/db

dentro del contenedor.

Despliegue del contenedor de MongoDB

Se creó el contenedor utilizando la imagen oficial de MongoDB 8.0:

```
podman run -d --name mongo-db -p 27017:27017 -v mongodb_data:/data/db mongo:8.0
```

```
PS C:\Users\mplf2> podman run -d --name mongo-db -p 27017:27017 -v mongodb_data:/data/db mongo:8.0
f24ae0ee710656e996073ef9311a637e448f6d4df69cfa7331474ab9ac95743b
```

Desglose de los parámetros empleados

- -d: ejecuta el contenedor en segundo plano.
- --name mongo-db: asigna el nombre mongo-db al contenedor.
- -p 27017:27017: mapea el puerto 27017 del equipo al puerto 27017 del contenedor.

- -v mongodb_data:/data/db: monta el volumen mongodb_data en el directorio donde MongoDB almacena sus datos.
- mongo:8.0: utiliza la imagen de MongoDB versión 8.0.

Después de crear el contenedor se verificó su estado mediante:

podman ps

El resultado obtenido fue:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
f24ae0ee7106	docker.io/library/mongo:8.0	mongod	7 seconds ago	Up 8 seconds	0.0.0.0:27017->27017/tcp	mongo-db

Verificación del funcionamiento de MongoDB

Para comprobar que MongoDB estaba funcionando dentro del contenedor, se accedió a la consola interactiva mediante:

podman exec -it mongo-db mongosh

```
PS C:\Users\mplf2> podman exec -it mongo-db mongosh
Current Mongosh Log ID: 6ab4b89f9e2d03439c91f97d
Connecting to:
  mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.11.1
Using MongoDB:
  8.0.32
Using Mongosh:
  2.11.1

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

To help improve our products, pseudo-anonymous usage data is collected from interactive and agentic sessions only, and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2026-09-24T05:43:36.824+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/production-notes-filesystem
2026-09-24T05:43:37.116+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-09-24T05:43:37.116+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-09-24T05:43:37.116+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-09-24T05:43:37.116+00:00: We suggest setting the contents of sysfsFile to 0.
2026-09-24T05:43:37.116+00:00: We suggest setting swappiness to 0 or 1, as swapping can cause performance problems.
-----
```

Posteriormente se ejecutó una prueba de comunicación con el servidor

```
test> db.runCommand({ ping: 1 })
{ ok: 1 }
```

Este resultado confirmó que el servidor MongoDB estaba activo y podía recibir y procesar comandos correctamente.

Inserción de datos iniciales

Una vez comprobado el funcionamiento de MongoDB, se seleccionó una base de datos de prueba, posteriormente se insertó un registro en una colección:

```
test> use mi_base
switched to db mi_base
mi_base> db.prueba.insertOne({
|   nota: "Dato guardado antes de apagar"
| })
{
  acknowledged: true,
  insertedId: ObjectId('6ab4bd7db52d77e9eca8eb91')
}
mi_base> |
```

El valor acknowledged: true confirma que la operación de inserción fue aceptada correctamente.

Para comprobar el registro almacenado:

```
mi_base> db.prueba.find()
[
  {
    _id: ObjectId('6ab4bd7db52d77e9eca8eb91'),
    nota: 'Dato guardado antes de apagar'
  }
]
mi_base> |
```

El documento insertado deberá aparecer en los resultados.

Prueba de persistencia de datos

Una de las principales finalidades del volumen es garantizar que los datos permanezcan almacenados, aunque el contenedor sea detenido.

Primero se salió de la consola de MongoDB:

```
test> use mi_base
switched to db mi_base
mi_base> db.prueba.insertOne({
  |   nota: "Dato guardado antes de apagar"
  | })
{
  acknowledged: true,
  insertedId: ObjectId('6ab4bd7db52d77e9eca8eb91')
}
mi_base> db.prueba.find()
[
  {
    _id: ObjectId('6ab4bd7db52d77e9eca8eb91'),
    nota: 'Dato guardado antes de apagar'
  }
]
mi_base> exit
PS C:\Users\mplf2> |
```

Posteriormente se detuvo el contenedor y se comprobó que el contenedor ya no estuviera ejecutándose:

```
PS C:\Users\mplf2> podman stop mongo-db
mongo-db
PS C:\Users\mplf2> podman ps
CONTAINER ID  IMAGE  COMMAND  CREATED  STATUS  PORTS  NAMES
PS C:\Users\mplf2> |
```

Aunque el contenedor estaba detenido, el volumen continuó existiendo. Esto se comprobó mediante:

```
PS C:\Users\mplf2> podman volume ls
DRIVER  VOLUME NAME
local   9785e2eafb6bbceb963d7422dc1aefa5e2a0c10917b04e38014ce40f33b484b1
local   bf73d0defcfc106a9fc2aff7881e45179f2ed638aeae20bc279b141caeca2e55d
local   a8e0c7d01b40d9b876302a118eda622195b65e5d9b71d4893d06a38600d1b74f
local   mongo_data
local   e1eb01594fda9e0f67f5a7c7e76eadcce80e443a791572380d3129e98b97394d
local   6a637d83e289973c7efa5451ca8e1f60a9e363841b40698d05b5242636cc9736
local   620b53938c281ff8a91c533ae6846aceb14ce7197425c4ca2d4b28b73db6e315
local   mongodb_data
local   0e14f09e47a9b2d225bf24396f35c0fb03cd4169f0d1e257cff032e918c7e962
PS C:\Users\mplf2> |
```

continuó apareciendo en la lista.

Posteriormente se inició nuevamente el contenedor y se comprobó su estado:

```
PS C:\Users\mplf2> podman start mongo-db
mongo-db
PS C:\Users\mplf2> podman ps
CONTAINER ID  IMAGE  COMMAND  CREATED  STATUS  PORTS  NAMES
f24ae0ee7106  docker.io/library/mongo:8.0  mongod  26 minutes ago  Up 12 seconds  0.0.0.0:27017->27017/tcp  mongo-db
```

Finalmente se accedió nuevamente a MongoDB, se seleccionó la base de datos y se consultaron los datos:

```
PS C:\Users\mplf2> podman exec -it mongo-db mongosh
Current Mongosh Log ID: 6ab4becc9c6352fccb47152a
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.11.1
Using MongoDB:      8.0.32
Using Mongosh:       2.11.1
mongosh 2.12.0 is available for download: https://www.mongodb.com/try/download/shell
For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-09-24T06:09:45.151+08:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/production-notes-filesystem
2026-09-24T06:09:45.647+08:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-09-24T06:09:45.647+08:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-09-24T06:09:45.647+08:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-09-24T06:09:45.647+08:00: We suggest setting the contents of sysfsFile to 0.
2026-09-24T06:09:45.647+08:00: We suggest setting swappiness to 0 or 1, as swapping can cause performance problems.
-----
test> use mi_base
switched to db mi_base
mi_base> db.prueba.find()
[
  {
    _id: ObjectId('6ab4bd7db52d77e9eca8eb91'),
    nota: 'Dato guardado antes de apagar'
  }
]
mi_base> |
```

El registro previamente almacenado deberá aparecer nuevamente, la información permaneció almacenada después de detener y volver a iniciar el contenedor, demostrando el funcionamiento del volumen persistente.

Conclusión

El despliegue de MongoDB mediante Podman en Windows se realizó correctamente utilizando la imagen mongo:8.0 y un volumen persistente administrado por Podman denominado mongodb_data.

Las pruebas realizadas permitieron comprobar que el contenedor puede iniciar correctamente, que MongoDB responde a las consultas realizadas mediante mongosh y que los datos pueden almacenarse y recuperarse posteriormente.

La prueba de persistencia demuestra que los datos almacenados en MongoDB no dependen exclusivamente del ciclo de vida del contenedor, ya que permanecen disponibles después de detener y volver a iniciar el servicio.